

Subset Sums Solved

Difficulty: **Medium** Accuracy: **72.55%** Submissions: **197K+** Points: **4**

Given an array **arr** of integers, return the sums of all subsets in the list. Return the sums in any order.

Examples:

Input: arr[] = [2, 3]

Output: [0, 2, 3, 5]

Explanation: When no elements are taken then Sum = 0. When only 2 is taken then Sum = 2. When only 3 is taken then Sum = 3. When elements 2 and 3 are taken then Sum = 2+3 = 5.

Input: arr[] = [1, 2, 1]

Output: [0, 1, 1, 2, 2, 3, 3, 4]

Explanation: The possible subset sums are 0 (no elements), 1 (either of the 1's), 2 (the element 2), and their combinations.

Input: arr[] = [5, 6, 7]

Output: [0, 5, 6, 7, 11, 12, 13, 18]

Explanation: The possible subset sums are 0 (no elements), 5, 6, 7, and their combinations.

```

1 class Solution {
2 public:
3     void subset(int ind, vector<int>& ds, vector<int>& arr, int n, vector<int>& ans) {
4         if (ind == n) {
5             int sum = 0;
6
7             for (auto it : ds) {
8                 sum += it;
9             }
10
11             ans.push_back(sum);
12             return;
13         }
14
15         ds.push_back(arr[ind]);
16         subset(ind + 1, ds, arr, n, ans);
17
18         ds.pop_back();
19         subset(ind + 1, ds, arr, n, ans);
20     }
21
22     vector<int> subsetSums(vector<int>& arr) {
23         vector<int> ds;
24         vector<int> ans;
25         int n = arr.size();
26
27         subset(0, ds, arr, n, ans);
28
29         return ans;
30     }
31 };

```

L7. All Kind of Patterns in Recursion | Print All | Print one | Count

2.10

```
f(i, ds, s)
{
    if (i == n)
    {
        if (s == sum)
            print(ds)
        return;
    }
}
```

```
ds.add(arr[i]);
s += arr[i];
f(i+1, ds, s);
ds.remove(arr[i]);
s -= arr[i];
```

```
f(i+1, ds, s)
```

```
}
```



8:18 / 34:11

Pseudo Code >



L7. All Kind of Patterns in Recursion | Print All | Print one | Count

```
1  if(s == sum) {
2      for(auto it : ds) cout << it << " ";
3      cout << endl;
4  }
5  return;
6  }
7
8  ds.push_back(arr[ind]);
9  s += arr[ind];
10
11 printS(ind+1, ds, s, sum, arr, n);
12
13 s -= arr[ind];
14 ds.pop_back();
15
16 // not pick
17 printS(ind+1, ds, s, sum, arr, n);
18 }
19
20 int main() {
21     #ifndef ONLINE_JUDGE
22     freopen("input.txt", "r", stdin);
23     freopen("output.txt", "w", stdout);
24     #endif
25     int arr[] = {1, 2, 1};
26     int n = 3;
27     int sum = 2;
28     vector<int> ds;
29     printS(0, ds, 0, sum, arr, n);
30
31     return 0;
32 }
```

input.txt

```
1 5
2 1 2 3 4 5
```

output.txt

```
1 1 1
2 2
3 1
```

[Finished in 1.9s]



L7. All Kind of Patterns in Recursion | Print All | Print one | Count

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 bool flag = false;
5 void printS(int ind, vector<int> &ds, int s, int sum, int arr[], int n) {
6     if(ind == n) {
7         if(s == sum && flag == false) {
8             flag = true;
9             for(auto it : ds) cout << it << " ";
10            cout << endl;
11        }
12        return;
13    }
14
15    ds.push_back(arr[ind]);
16    s += arr[ind];
17
18    printS(ind+1, ds, s, sum, arr, n);
19
20    s -= arr[ind];
21    ds.pop_back();
22
23    // not pick
24    printS(ind+1, ds, s, sum, arr, n);
25 }
26 int main() {
27     #ifndef ONLINE_JUDGE
28     freopen("input.txt", "r", stdin);
29     freopen("output.txt", "w", stdout);
30     #endif
31     int arr[] = {1, 2, 1};
32     int n = 3;
33     int sum = 2;
34     vector<int> ds;
35     printS(0, ds, 0, sum, arr, n);
36 }
```

input.txt

```
1 5
2 1 2 3 4 5
```

output.txt

```
1 1 1
2
```



[Finished in 1.9s]

12:34 / 34:11

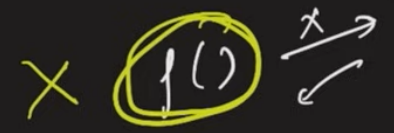
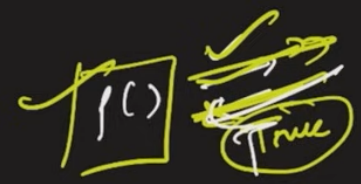
Modify >



(Technique to print
One answer)

f()

{



}

for f()

{

base case

cond $\rightarrow$ satisfied
return true
 $\rightarrow$ X set
return false

if (f() == true)
return true;

f()

return false



```

3
4
5 bool printS(int ind, vector<int> &ds, int s, int sum, int arr[], int n) {
6     if(ind == n) {
7         // condition satisfied
8         if(s == sum) {
9             for(auto it : ds) cout << it << " ";
10            cout << endl;
11            return true;
12        }
13        // condition not satisfied
14        else return false;
15    }
16
17    ds.push_back(arr[ind]);
18    s += arr[ind];
19
20    if(printS(ind+1, ds, s, sum, arr, n) == true) {
21        return true;
22    }
23
24    s -= arr[ind];
25    ds.pop_back();
26
27    // not pick
28    if(printS(ind+1, ds, s, sum, arr, n) == true) return true;
29
30    return false;
31 }
32
33 int main() {
34     #ifndef ONLINE_JUDGE
35     freopen("input.txt", "r", stdin);
36     freopen("output.txt", "w", stdout);
37     #endif
38     int arr[] = {1, 2, 1};
39     int sum = 2;

```

input.txt

```

1 5
2 1 2 3 4 5

```

output.txt

```

1 1 1
2

```

[Finished in 2.2s]



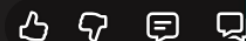
2.10

→ Count the subsequences with sum = 12

{1, 2, 1}

$k = 2$

(2 subsequences)



23:53 / 34:11

Modify >



2.10

mt $f()$
{

base case

subn 1 $\rightarrow$ condition satisfies

subn 0 $\rightarrow$ condition not satisfies

$l = f()$

$r = f()$

subn $l+r$;

}



base case

subn 1 $\rightarrow$ condition satisfies

subn 0 $\rightarrow$ condition not satisfies

$\left\{ \begin{array}{l} l = f() \\ r = g() \end{array} \right\}^2$

subn $l+r$;

}

$s = 0$
 $fn(i = 1 \rightarrow n)$
 $\{ s += f(); \}$ } N times

subn s



47. All Kind of Patterns in Recursion | Print All | Print one | Count

```

5 int printS(int ind, int s, int sum, int arr[], int n) {
6     if(ind == n) {
7         // condition satisfied
8         if(s == sum) return 1;
9         // condition not satisfied
10        else return 0;
11    }
12
13
14    s += arr[ind];
15
16    int l = printS(ind+1, s, sum, arr, n) ;
17
18    s -= arr[ind];
19
20
21    // not pick
22    int r = printS(ind+1, s, sum, arr, n);
23
24    return l + r;
25 }
26 int main() {
27     #ifndef ONLINE_JUDGE
28     freopen("input.txt", "r", stdin);
29     freopen("output.txt", "w", stdout);
30     #endif
31     int arr[] = {1, 2, 1};
32     int n = 3;
33     int sum = 2;
34     cout << printS(0, 0, sum, arr, n);
35
36     return 0;
37 }

```

output.txt

1 2

[Finished in 0.8s]



$f(0,0)$
 $\{$
 $\hat{y}()x$

$f(1,1)$
 $\{$
 $\hat{y}()x$

$\{$
 $\hat{y}()x$

$l = f(1,1)$ ✓

$l = f(2,3)$ ✓

$x = f(2,1)$ ✗

$l = f(2,3)$ ✓

$x = f(2,3)$ ✓

return $l + x$;

}

$x = f(1,0)$ ✗
return $l + x$;
}

return $l + x$;

}

$f(2,1)$
 $\{$
 $\hat{y}()x$

$l = f(2,2)$ ✓



print

→ pattern

print 1

→ not T/F &
avoid function

Recursion calls
if you get true

Count

→

sub 1
sub 0

add all f()

& sub

